ARTIFICIAL INTELLIGENCE
SEM-III

GUIDELINE PRACTICALS

NAME: Pratyush Pravanjan Pradhan
COURSE: B.Sc. (H) Computer Science
ROLL No. : 251248

1. Write a PROLOG program to implement the family tree and demonstrate the family relationship.

```
male(pratyush).
male(abhi).
male(akshat).

female(sami).
female(gauri).
female(rani).

parent(pratyush,abhi).
parent(sami,abhi).

parent(abhi,akshat).
parent(gauri,akshat).
parent(abhi,rani).
parent(gauri,rani).

father(X,Y):-
    male(X),
    parent(X,Y).
mother(X,Y):-
    female(X),
    parent(X,Y).
brother(X,Y):-
    male(X),
    parent(P,X),
    parent(P,Y),
    X \= Y.
sister(X,Y):-
    female(X),
    parent(P,X),
    parent(P,Y),
    X \= Y.
grandparent(X,Y):-
    parent(X,Z),
    parent(Z,Y).
grandfather(X,Y):-
    male(X),
    grandparent(X,Y).
grandmother(X,Y):-
    female(X),
    grandparent(X,Y).
```

```
| ?- father(pratyush,abhi).

yes
| ?- mother(sami,abhi).

(1 ms) yes
| ?- brother(akshat,abhi).

no
| ?- sister(rani,akshat).

true ?

yes
| ?- grandfather(pratyush,akshat).

true ?

yes
| ?- grandmother(sami,akshat).

true ?

yes
```

```
{trace}
| ?- mother(sami,abhi).
      1    1  Call: mother(sami,abhi) ?
      2    2  Call: female(sami) ?
      2    2  Exit: female(sami) ?
      3    2  Call: parent(sami,abhi) ?
      3    2  Exit: parent(sami,abhi) ?
      1    1  Exit: mother(sami,abhi) ?

(1 ms) yes
{trace}
| ?- grandfather(X,akshat).
      1    1  Call: grandfather(_23,akshat) ?
      2    2  Call: male(_23) ?
      2    2  Exit: male(pratyush) ?
      3    2  Call: grandparent(pratyush,akshat) ?
      4    3  Call: parent(pratyush,_140) ?
      4    3  Exit: parent(pratyush,abhi) ?
      5    3  Call: parent(abhi,akshat) ?
      5    3  Exit: parent(abhi,akshat) ?
      3    2  Exit: grandparent(pratyush,akshat) ?
      1    1  Exit: grandfather(pratyush,akshat) ?

X = pratyush ?

(1 ms) yes
{trace}
```

2. Write a PROLOG program to implement conc(L1, L2, L3) where L2 is the list to be appended with L1 to get the resulted list L3.

```
goconcat:-
  write('Enter first list:'),read(L1),
  write('Enter second list: '),read(L2),
  concat(L1,L2,R),
  write('the result is: '),write(R).


concat([],L2,L2).
concat([H|T],L2,[H|R]):-
  concat(T,L2,R).
```

```
| ?- goconcat.
Enter first list:[1,2,3,5].
Enter second list: [4,6,7,8].
the result is: [1,2,3,5,4,6,7,8]
```

```
{trace}
| ?- concat([1,2,3,5],[4,6,7,8],R).
      1    1  Call: concat([1,2,3,5],[4,6,7,8],_39) ?
      2    2  Call: concat([2,3,5],[4,6,7,8],_72) ?
      3    3  Call: concat([3,5],[4,6,7,8],_99) ?
      4    4  Call: concat([5],[4,6,7,8],_126) ?
      5    5  Call: concat([],[4,6,7,8],_153) ?
      5    5  Exit: concat([],[4,6,7,8],[4,6,7,8]) ?
      4    4  Exit: concat([5],[4,6,7,8],[5,4,6,7,8]) ?
      3    3  Exit: concat([3,5],[4,6,7,8],[3,5,4,6,7,8]) ?
      2    2  Exit: concat([2,3,5],[4,6,7,8],[2,3,5,4,6,7,8]) ?
      1    1  Exit: concat([1,2,3,5],[4,6,7,8],[1,2,3,5,4,6,7,8]) ?

R = [1,2,3,5,4,6,7,8]

(1 ms) yes
```

3. Write a PROLOG program to implement reverse(L, R) where List L is original and List R is reversed list.

```
1  gorev:-
2    write('Enter the list: '),read(L),
3    reversed(L,R),
4    write('The reversed list is: '),write(R).
5
6  reversed(L,R):-
7    reverse_ac(L,[],R).
8
9  reverse_ac([],Ac,Ac).
10
11 reverse_ac([H|T],Ac,R):-
12   reverse_ac(T,[H|Ac],R).
 ~
```

```
| ?- gorev.
Enter the list: [1,2,3].
The reversed list is: [3,2,1]

yes
```

```
{trace}
| ?- reversed([1,2,3],R).
      1    1  Call: reversed([1,2,3],_29) ?
      2    2  Call: reverse_ac([1,2,3],[],_29) ?
      3    3  Call: reverse_ac([2,3],[1],_29) ?
      4    4  Call: reverse_ac([3],[2,1],_29) ?
      5    5  Call: reverse_ac([],[3,2,1],_29) ?
      5    5  Exit: reverse_ac([],[3,2,1],[3,2,1]) ?
      4    4  Exit: reverse_ac([3],[2,1],[3,2,1]) ?
      3    3  Exit: reverse_ac([2,3],[1],[3,2,1]) ?
      2    2  Exit: reverse_ac([1,2,3],[],[3,2,1]) ?
      1    1  Exit: reversed([1,2,3],[3,2,1]) ?

R = [3,2,1]

yes
```

4. Write a PROLOG program to calculate the sum of two numbers.

```
gosum:-
  write('Enter the first number: '),read(X),
  write('Enter the second number: '),read(Y),
  write('the sum is: '),
  summ(X,Y).


summ(X,Y):-
  W is X + Y,
  write(W).
```

```
| ?- gosum.
Enter the first number: 23.
Enter the second number: 3.
the sum is: 26
```

```
{trace}
| ?- summ(23,3).
      1    1  Call: summ(23,3) ?
      2    2  Call: _89 is 23+3 ?
      2    2  Exit: 26 is 23+3 ?
      3    2  Call: write(26) ?
26
      3    2  Exit: write(26) ?
      1    1  Exit: summ(23,3) ?

yes
```

5. Write a PROLOG program to implement max(X, Y, M) so that M is the maximum of two numbers X and Y.

```
1  gomax:-
2    write('enter the first number: '),read(X),
3    write('enter the second number: '),read(Y),
4    maxm(X,Y,M),
5    write('the largest number is: '),write(M).
6
7  maxm(X,Y,M):-
8    M is X, X>Y,!;M is Y, Y>X.
```

```
| ?- gomax.
enter the first number: 23.
enter the second number: 12.
the largest number is: 23
```

```
{trace}
| ?- maxm(23,12,M).
      1    1  Call: maxm(23,12,_23) ?
      2    2  Call: _23 is 23 ?
      2    2  Exit: 23 is 23 ?
      3    2  Call: 23>12 ?
      3    2  Exit: 23>12 ?
      1    1  Exit: maxm(23,12,23) ?

M = 23

(1 ms) yes
```

6. Write a program in PROLOG to implement factorial (N, F) where F represents the factorial of a number N.

```
gofac:-
  write('enter the number: '),read(N),
  fact(N,F),
  write('the factorial is: '),write(F).


fact(0,1).
fact(N,F):-
  N>0,
  N1 is N-1,
  fact(N1,F1),
  F is F1*N.
```

```
| ?- gofac.
enter the number: 3.
the factorial is: 6

true ?

yes
```

```
{trace}
| ?- fact(3,F).
      1    1  Call: fact(3,_23) ?
      2    2  Call: 3>0 ?
      2    2  Exit: 3>0 ?
      3    2  Call: _120 is 3-1 ?
      3    2  Exit: 2 is 3-1 ?
      4    2  Call: fact(2,_145) ?
      5    3  Call: 2>0 ?
      5    3  Exit: 2>0 ?
      6    3  Call: _197 is 2-1 ?
      6    3  Exit: 1 is 2-1 ?
      7    3  Call: fact(1,_222) ?
      8    4  Call: 1>0 ?
      8    4  Exit: 1>0 ?
      9    4  Call: _274 is 1-1 ?
      9    4  Exit: 0 is 1-1 ?
     10    4  Call: fact(0,_299) ?
     10    4  Exit: fact(0,1) ?
     11    4  Call: _327 is 1*1 ?
     11    4  Exit: 1 is 1*1 ?
      7    3  Exit: fact(1,1) ?
     12    3  Call: _356 is 1*2 ?
     12    3  Exit: 2 is 1*2 ?
      4    2  Exit: fact(2,2) ?
     13    2  Call: _23 is 2*3 ?
     13    2  Exit: 6 is 2*3 ?
      1    1  Exit: fact(3,6) ?

F = 6 ?

(1 ms) yes
```

7. Write a program in PROLOG to implement generate_fib(N,T) where T represents the Nth term of the Fibonacci series.

```
gofib:-
   write('enter the value for n: '),read(N),
   fib(N,R),
   write('the nth term is: '),
   write(R).

fib(1, 0).
fib(2, 1).
fib(N, Result) :-
   N > 2,
   N1 is N - 1,
   N2 is N - 2,
   fib(N1, F1),
   fib(N2, F2),
   Result is F1 + F2.
```

```
| ?- gofib.
enter the value for n: 3.
the nth term is: 1
```

```
| ?- gofib.
enter the value for n: 5.
the nth term is: 3
```

```
{trace}
| ?- fib(3,R).
      1    1  Call: fib(3,_23) ?
      2    2  Call: 3>2 ?
      2    2  Exit: 3>2 ?
      3    2  Call: _120 is 3-1 ?
      3    2  Exit: 2 is 3-1 ?
      4    2  Call: _148 is 3-2 ?
      4    2  Exit: 1 is 3-2 ?
      5    2  Call: fib(2,_173) ?
      5    2  Exit: fib(2,1) ?
      6    2  Call: fib(1,_198) ?
      6    2  Exit: fib(1,0) ?
      7    2  Call: _23 is 1+0 ?
      7    2  Exit: 1 is 1+0 ?
      1    1  Exit: fib(3,1) ?

R = 1 ?
```

8. Write a PROLOG program to implement power (Num, Pow, Ans) : where Num is raised to the power Pow to get Ans.

```
gopow:-
   write('enter the base: '),read(B),
   write('enter the exponent: '),read(P),
   power(B,P,R),
   write('the value is: '),write(R).

power(_, 0, 1) :- !.
power(Base, Exp, Res) :-
   Exp > 0,
   NxtE is Exp - 1,
   power(Base, NxtE, SubRes),
   Res is Base * SubRes.
```

```
| ?- gopow.
enter the base: 3.
enter the exponent: 2.
the value is: 9

yes
```

```
{trace}
| ?- power(3,2,R).
      1    1  Call: power(3,2,_23) ?
      2    2  Call: 2>0 ?
      2    2  Exit: 2>0 ?
      3    2  Call: _121 is 2-1 ?
      3    2  Exit: 1 is 2-1 ?
      4    2  Call: power(3,1,_147) ?
      5    3  Call: 1>0 ?
      5    3  Exit: 1>0 ?
      6    3  Call: _199 is 1-1 ?
      6    3  Exit: 0 is 1-1 ?
      7    3  Call: power(3,0,_225) ?
      7    3  Exit: power(3,0,1) ?
      8    3  Call: _253 is 3*1 ?
      8    3  Exit: 3 is 3*1 ?
      4    2  Exit: power(3,1,3) ?
      9    2  Call: _23 is 3*3 ?
      9    2  Exit: 9 is 3*3 ?
      1    1  Exit: power(3,2,9) ?

R = 9

(1 ms) yes
```

9. PROLOG program to implement multi (N1, N2, R) : where N1 and N2 denotes the numbers to be multiplied and R represents the result.

```
gomult:-
   write('enter the first number: '),read(X),
   write('enter the second number: '),read(Y),
   write('the product is: '),mult(X,Y,A),
   write(A).

mult(X,Y,A):-  A is X * Y.
```

```
| ?- gomult.
enter the first number: 2.
enter the second number: 34.
the product is: 68

yes
```

```
{trace}
| ?- power(3,2,R).
      1    1  Call: power(3,2,_23) ?
      2    2  Call: 2>0 ?
      2    2  Exit: 2>0 ?
      3    2  Call: _121 is 2-1 ?
      3    2  Exit: 1 is 2-1 ?
      4    2  Call: power(3,1,_147) ?
      5    3  Call: 1>0 ?
      5    3  Exit: 1>0 ?
      6    3  Call: _199 is 1-1 ?
      6    3  Exit: 0 is 1-1 ?
      7    3  Call: power(3,0,_225) ?
      7    3  Exit: power(3,0,1) ?
      8    3  Call: _253 is 3*1 ?
      8    3  Exit: 3 is 3*1 ?
      4    2  Exit: power(3,1,3) ?
      9    2  Call: _23 is 3*3 ?
      9    2  Exit: 9 is 3*3 ?
      1    1  Exit: power(3,2,9) ?

R = 9

(1 ms) yes
```

10. Write a PROLOG program to implement memb(X, L): to check whether X is a member of L or not.

```
1  gomemb:-
2     write('enter the element: '),
3     read(X),
4     write('write the list: '),
5     read(L),
6     write('is this present:'),
7     is_memb(X,L).
8
9  is_memb(A,[A|_]):-!.
10 is_memb(A,[_|T]):-
11    is_memb(A,T).
```

```
| ?- gomemb.
enter the element: 12.
write the list: [1,2,12,5].
is this present:

yes
```

```
| ?- gomemb.
enter the element: 12.
write the list: [1,2,3,5].
is this present:

(1 ms) no
```

```
{trace}
| ?- is_memb(12,[1,2,12,5]).
      1    1  Call: is_memb(12,[1,2,12,5]) ?
      2    2  Call: is_memb(12,[2,12,5]) ?
      3    3  Call: is_memb(12,[12,5]) ?
      3    3  Exit: is_memb(12,[12,5]) ?
      2    2  Exit: is_memb(12,[2,12,5]) ?
      1    1  Exit: is_memb(12,[1,2,12,5]) ?

yes
{trace}
```

11. Write a PROLOG program to implement sumlist(L, S) so that S is the sum of a given list L.

```
gosuml:-
   write('enter the list: '),read(X),
   sumlst(X,S),
   write('sum of the list is:'),write(S).

sumlst([],0).
sumlst([H|T],S):-
   sumlst(T,R1),
   S is H + R1.
```

```
| ?- gosuml.
enter the list: [1,2,3,4].
sum of the list is:10

yes
```

```
{trace}
| ?- sumlst([1,2,3,4],S).
      1    1  Call: sumlst([1,2,3,4],_31) ?
      2    2  Call: sumlst([2,3,4],_100) ?
      3    3  Call: sumlst([3,4],_124) ?
      4    4  Call: sumlst([4],_148) ?
      5    5  Call: sumlst([],_172) ?
      5    5  Exit: sumlst([],0) ?
      6    5  Call: _200 is 4+0 ?
      6    5  Exit: 4 is 4+0 ?
      4    4  Exit: sumlst([4],4) ?
      7    4  Call: _229 is 3+4 ?
      7    4  Exit: 7 is 3+4 ?
      3    3  Exit: sumlst([3,4],7) ?
      8    3  Call: _258 is 2+7 ?
      8    3  Exit: 9 is 2+7 ?
      2    2  Exit: sumlst([2,3,4],9) ?
      9    2  Call: _31 is 1+9 ?
      9    2  Exit: 10 is 1+9 ?
      1    1  Exit: sumlst([1,2,3,4],10) ?

S = 10

yes
```

12. Write a PROLOG program to implement two predicates evenlength(List) and oddlength(List) so that they are true if their argument is a list of even or odd length respectively.

```
goeos:-
   write('enter the list: '),
   read(L),
   evenlen(L).

evenlen([]):-write('the list is even.'),!.
evenlen([_]):-write('the list is odd.'),!.
evenlen([_|T]):-oddlen(T).
oddlen([_|T]):-evenlen(T).
```

```
| ?- goeos.
enter the list: [1,2,3].
the list is odd.

yes
```

```
| ?- goeos.
enter the list: [1,2,3,4].
the list is even.

yes
```

```
{trace}
| ?- evenlen([1,2,3]).
      1    1  Call: evenlen([1,2,3]) ?
      2    2  Call: oddlen([2,3]) ?
      3    3  Call: evenlen([3]) ?
      4    4  Call: write('the list is odd.') ?
the list is odd.
      4    4  Exit: write('the list is odd.') ?
      3    3  Exit: evenlen([3]) ?
      2    2  Exit: oddlen([2,3]) ?
      1    1  Exit: evenlen([1,2,3]) ?

yes
{trace}
```

```
| ?- evenlen([1,2,3,4]).
      1    1  Call: evenlen([1,2,3,4]) ?
      2    2  Call: oddlen([2,3,4]) ?
      3    3  Call: evenlen([3,4]) ?
      4    4  Call: oddlen([4]) ?
      5    5  Call: evenlen([]) ?
      6    6  Call: write('the list is even.') ?
the list is even.
      6    6  Exit: write('the list is even.') ?
      5    5  Exit: evenlen([]) ?
      4    4  Exit: oddlen([4]) ?
      3    3  Exit: evenlen([3,4]) ?
      2    2  Exit: oddlen([2,3,4]) ?
      1    1  Exit: evenlen([1,2,3,4]) ?

yes
{trace}
```

13. Write a PROLOG program to implement maxlist(L, M) so that M is the maximum number in the list.

```
gomaxl:-
    write('enter the list:'),
    read(L),
    maxlist(L,M),
    write('Max element:'),
    write(M),nl.


maxlist([X],X).
maxlist([H|T],M):-
    maxlist(T,M1),
    M is max(H,M1).
```

```
| ?-  gomaxl.
enter the list:[23,13,12,1].
Max element:23

true ?

yes
```

```
{trace}
| ?- maxlist([23,13,12,1],M).
      1    1  Call: maxlist([23,13,12,1],_31) ?
      2    2  Call: maxlist([13,12,1],_100) ?
      3    3  Call: maxlist([12,1],_124) ?
      4    4  Call: maxlist([1],_148) ?
      4    4  Exit: maxlist([1],1) ?
      5    4  Call: _176 is max(12,1) ?
      5    4  Exit: 12 is max(12,1) ?
      3    3  Exit: maxlist([12,1],12) ?
      6    3  Call: _205 is max(13,12) ?
      6    3  Exit: 13 is max(13,12) ?
      2    2  Exit: maxlist([13,12,1],13) ?
      7    2  Call: _31 is max(23,13) ?
      7    2  Exit: 23 is max(23,13) ?
      1    1  Exit: maxlist([23,13,12,1],23) ?

M = 23 ?

(1 ms) yes
{trace}
```

14. Write a PROLOG program to implement insert(I, N, L, R) that inserts an item I into Nth position of list L to generate a list R.

```
goins:-
   write('Enter the list:'),
   read(L),
   write('Enter the position:'),
   read(N),
   write('Enter the element to be inserted:'),
   read(I),
   insert_nth(I,N,L,R),
   write('The resultant list is:'),
   write(R).



insert_nth(I,1,T,[I|T]):-!.
insert_nth(I,N,[Y|T],[Y|T1]):-
   N>1,
   N1 is N-1,
   insert_nth(I,N1,T,T1).
```

```
| ?- goins.
Enter the list:[1,2,3,4,5].
Enter the position:3.
Enter the element to be inserted:6.
The resultant list is:[1,2,6,3,4,5]

yes
```

```
{trace}
| ?- insert_nth(6,3,[1,2,3,4,5],R).
      1    1  Call: insert_nth(6,3,[1,2,3,4,5],_33) ?
      2    2  Call: 3>1 ?
      2    2  Exit: 3>1 ?
      3    2  Call: _134 is 3-1 ?
      3    2  Exit: 2 is 3-1 ?
      4    2  Call: insert_nth(6,2,[2,3,4,5],_67) ?
      5    3  Call: 2>1 ?
      5    3  Exit: 2>1 ?
      6    3  Call: _215 is 2-1 ?
      6    3  Exit: 1 is 2-1 ?
      7    3  Call: insert_nth(6,1,[3,4,5],_148) ?
      7    3  Exit: insert_nth(6,1,[3,4,5],[6,3,4,5]) ?
      4    2  Exit: insert_nth(6,2,[2,3,4,5],[2,6,3,4,5]) ?
      1    1  Exit: insert_nth(6,3,[1,2,3,4,5],[1,2,6,3,4,5]) ?

R = [1,2,6,3,4,5]

yes
{trace}
```

15. Write a PROLOG program to implement delete(N, L, R) that removes the element on Nth position from a list L to generate a list R.

```
godel:-
   write('Enter the list:'),
   read(L),
   write('Enter the position:'),
   read(N),
   deleten(N,L,R),
   write('The resultant list is:'),
   write(R).


deleten(1,[_|T],T):-!.
deleten(N,[Y|T],[Y|T1]):-
   N>1,
   N1 is N-1,
   deleten(N1,T,T1).
```

```
| ?- godel.
Enter the list:[1,2,3,4,5].
Enter the position:3.
The resultant list is:[1,2,4,5]

(1 ms) yes
```

```
{trace}
| ?- deleten(3,[1,2,3,4,5],R).
      1    1  Call: deleten(3,[1,2,3,4,5],_33) ?
      2    2  Call: 3>1 ?
      2    2  Exit: 3>1 ?
      3    2  Call: _133 is 3-1 ?
      3    2  Exit: 2 is 3-1 ?
      4    2  Call: deleten(2,[2,3,4,5],_66) ?
      5    3  Call: 2>1 ?
      5    3  Exit: 2>1 ?
      6    3  Call: _213 is 2-1 ?
      6    3  Exit: 1 is 2-1 ?
      7    3  Call: deleten(1,[3,4,5],_146) ?
      7    3  Exit: deleten(1,[3,4,5],[4,5]) ?
      4    2  Exit: deleten(2,[2,3,4,5],[2,4,5]) ?
      1    1  Exit: deleten(3,[1,2,3,4,5],[1,2,4,5]) ?

R = [1,2,4,5]

yes
```